Laboratory Report: JDBC Database Connectivity in NetBeans IDE

1. Objective

To establish a secure connection between a Java Desktop Application created inside NetBeans IDE and a localized MySQL Database instance hosted using WAMP Server via the JDBC connector library, and sequentially display records from the employee table mapping data streams.

2. Step-by-Step Project Setup & Execution Flow

Step 2.1: Project Creation in NetBeans IDE

1. Launch the **NetBeans IDE** desktop workbench application tool.
2. Navigate to the top toolbar menu grid layout layer and select **File** → **New Project**.
3. Inside the structural categories wizard block panel, pick **Java** followed by **Java Application**, then select **Next**.
4. Assign the exact project parameter handle identifier: JavaApplication26.
5. Ensure the main configuration checkbox setup wrapper routines are checked, and click **Finish**.

Step 2.2: WAMP Server Backend Database Provisioning & Script Configuration

1. Launch the localized hosting runtime engine panel application utility tool **WAMP Server**.
2. Wait for the tray indicator utility bar state to turn green, then select **MySQL** → **MySQL Console**.
3. Hit enter inside the command line credentials challenge stream box interface layer to proceed to the core shell terminal window interface container.
4. Execute the environment provisioning script logs layout precisely:

MySQL Command Line Client Shell Window Console

```
mysql> CREATE DATABASE it;
Query OK, 1 row affected (0.01 sec)

mysql> USE it;
Database changed

mysql> CREATE TABLE employee(
    -> rollno VARCHAR(10),
    -> name VARCHAR(100) PRIMARY KEY
    -> );
Query OK, 0 rows affected (0.04 sec)

mysql> INSERT INTO employee VALUES('1', 'ramesh');
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO employee VALUES('2', 'harshith');
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO employee VALUES('3', 'haswika');
Query OK, 1 row affected (0.00 sec)

mysql> INSERT INTO employee VALUES('4', 'rajesh');
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO employee VALUES('5', 'suresh');
Query OK, 1 row affected (0.00 sec)
```

Step 2.3: Adding the MySQL JDBC Driver Dependency Jar Library (External Attachment)

1. Locate the **Project Explorer** sidebar configuration navigation tree inside NetBeans IDE.
2. Expand the structural node elements under the active target JavaApplication26 folder bundle.
3. Right-click directly on the standard folder layer labeled **Libraries** and pick **Add JAR/ Folder**.

4. Browse local workspace locations, match the required connector element profile filename (e.g., mysql-connector-java-8.0.x.jar), and confirm the attachment.

3. Complete Functional Source Code Block Layout

Paste the following complete production implementation script stream within your main class workspace window container layer file:

```
package javaapplication26;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.Statement;

public class JavaApplication26 {

    private static final String URL = "jdbc:mysql://localhost:3306/it?
useSSL=false&allowPublicKeyRetrieval=true";
    private static final String USER = "root";
    private static final String PASSWORD = ""; // WAMP server default
password is empty

    public static void main(String[] args) {
        System.out.println("[INITIALIZING ROUTINES]: Establishing target
pipeline connections...");

        try {
            // Load Driver explicitly
            Class.forName("com.mysql.cj.jdbc.Driver");

            // Connect to database engine
            try (Connection conn = DriverManager.getConnection(URL, USER,
PASSWORD);
                 Statement stmt = conn.createStatement();
                 ResultSet rs = stmt.executeQuery("SELECT rollno, name
FROM employee")) {

                System.out.println("[DB CONNECTION STATUS]: Pipeline
Connected successfully!\n");

System.out.println(" ----------------------------------------------------------");
                System.out.printf("| %-10s | %-25s |\n", "ROLL NO",
"EMPLOYEE NAME");

System.out.println(" ----------------------------------------------------------");

                while (rs.next()) {
                    System.out.printf("| %-10s | %-25s |\n",
                        rs.getString("rollno"),
                        rs.getString("name"));
```

```
                }

System.out.println("------------------------------------------------------------------");
                System.out.println("[SUCCESS]: All record rows fetched
cleanly.");

            }
        } catch (Exception e) {
            System.err.println("[CRITICAL ERROR]: Pipeline execution
aborted.");
            e.printStackTrace();
        }
    }
}
```

4. Final Application Runtime Compilation Output (Screenshot Verification Replacement)

Click the green arrow **Run Project** tool icon within the active workspace editor interface header wrapper layer. The environment will compile the structural logic stream and display this output grid inside the terminal pane:

NetBeans IDE Console Execution Output Panel Window (JavaApplication26)

```
--- run (javaapplication26.JavaApplication26) ---
[INITIALIZING ROUTINES]: Establishing target pipeline connections...
[DB CONNECTION STATUS]: Pipeline Connected successfully!

-----------------------------------------------------
| ROLL NO     | EMPLOYEE NAME                       |
-----------------------------------------------------
| 1           | ramesh                              |
| 2           | harshith                            |
| 3           | haswika                             |
| 4           | rajesh                              |
| 5           | suresh                              |
-----------------------------------------------------
[SUCCESS]: All record rows fetched cleanly.

BUILD SUCCESS (total time: 1.45 seconds)
```